HACK THE BOX ACADEMY

API Attacks

Skills Assessment — Relatório de Teste de Intrusão (Writeup)

Módulo: API Attacks (HTB Academy)

Tipo de avaliação: API Penetration Test (REST / Swagger & OpenAPI)

Alvo: API v2 — Suppliers & Customers Platform

Data: Julho de 2026

Autor: Pentester HTB

```
HTB{ **** FLAG CAPTURADA — ver Seção 8 **** }
```

Sumário

1. Sumário Executivo
2. Escopo e Informações do Alvo
3. Metodologia e Ferramentas
4. Fase 1 — Reconhecimento via Swagger / OpenAPI
5. Fase 2 — Autenticação e Privilégios Excessivos (BFLA)
6. Fase 3 — Enumeração de Fornecedores (BOLA) e Vazamento de Perguntas de Segurança
7. Fase 4 — Ataque de Força Bruta na Resposta de Segurança
8. Fase 5 — Account Takeover via Reset de Senha
9. Fase 6 — Local File Inclusion via Upload de CV (URI Handler)
10. Resumo das Vulnerabilidades e Riscos
11. Recomendações de Correção
12. Conclusão

1. Sumário Executivo

Este documento apresenta a solução (writeup) desenvolvida para o Skills Assessment do módulo **API Attacks** da Hack The Box Academy. O objetivo do exercício era avaliar uma API REST (v2) de uma plataforma que conecta **Clientes (Customers)** e **Fornecedores (Suppliers)**, documentada via Swagger/OpenAPI, identificando falhas de autenticação, autorização e manipulação de arquivos que permitissem o acesso não autorizado a dados sensíveis e, por fim, a captura da flag da avaliação.

Ao longo do teste foram identificadas e exploradas em cadeia as seguintes classes de vulnerabilidades, mapeadas segundo o **OWASP API Security Top 10**:

- **API5:2023 — Broken Function Level Authorization (BFLA):** uma conta de cliente comum obteve papéis (roles) de fornecedor (`Suppliers_Get` , `Suppliers_GetAll`), permitindo acesso a funcionalidades que deveriam ser exclusivas de outro perfil de usuário.
- **API1:2023 — Broken Object Level Authorization (BOLA):** o endpoint de listagem de fornecedores expôs dados sensíveis de todos os registros, incluindo perguntas de segurança configuradas para recuperação de senha.
- **API2:2023 — Broken Authentication:** o endpoint de reset de senha por pergunta de segurança não possuía limitação de tentativas (rate limiting/lockout), permitindo um ataque de força bruta bem-sucedido e a tomada de controle (account takeover) de uma conta de fornecedor.
- **API8:2023 — Security Misconfiguration / Local File Inclusion:** o endpoint de atualização de perfil do fornecedor aceitava um parâmetro de URI para o currículo (CV) sem validação de esquema, permitindo a leitura de arquivos arbitrários do servidor através do esquema `file://` , o que resultou na exfiltração da flag do desafio.

A combinação dessas falhas demonstra um cenário clássico de **escalonamento de privilégios encadeado**: reconhecimento da API → abuso de autorização para enumerar dados → vazamento de informação sensível → força bruta direcionada → account takeover → leitura arbitrária de arquivos → exfiltração de dados confidenciais (flag).

2. Escopo e Informações do Alvo

Item	Detalhes
Alvo principal	`154.57.164.66:31488` / `154.57.164.75:30102`
Documentação da API	Swagger UI — `/swagger/index.html` (OpenAPI v2)
Grupos de recursos	Authentication, Customers, Products, Roles, Supplier-Companies, Suppliers
Esquema de autenticação	JWT Bearer Token (login separado para Customers e Suppliers)
Observação do desenvolvedor	A documentação do Swagger informava que a v1 continha "uma infinidade de vulnerabilidades" já corrigidas, e que a equipe de segurança estava confiante de que a v2 estaria livre de falhas — afirmação que se mostrou incorreta.

3. Metodologia e Ferramentas

A avaliação seguiu a metodologia padrão de teste de intrusão em APIs: reconhecimento da especificação, mapeamento de endpoints e papéis, testes de autorização (BOLA/BFLA), testes de autenticação e, por fim, testes de manipulação de entrada (injeção de esquemas de URI).

Ferramenta	Finalidade
Swagger UI	Leitura da especificação OpenAPI, execução manual de requisições ("Try it out")
Burp Suite	Interceptação, repetição e edição manual de requisições HTTP
ffuf	Ataque de força bruta (modo `clusterbomb`) combinando lista de e-mails e lista de respostas candidatas
CyberChef	Decodificação Base64 do conteúdo do arquivo exfiltrado
DeepSeek (LLM)	Apoio na filtragem/análise de um payload JSON extenso (lista de fornecedores)

4. Fase 1 — Reconhecimento via Swagger / OpenAPI

O primeiro passo consistiu em revisar a documentação Swagger exposta publicamente pela API, mapeando todos os grupos de endpoints disponíveis. O grupo **Authentication** chamou atenção por expor fluxos distintos de login e recuperação de senha para clientes e fornecedores, incluindo um mecanismo de reset baseado em *pergunta de segurança* — um padrão historicamente propenso a falhas de força bruta.

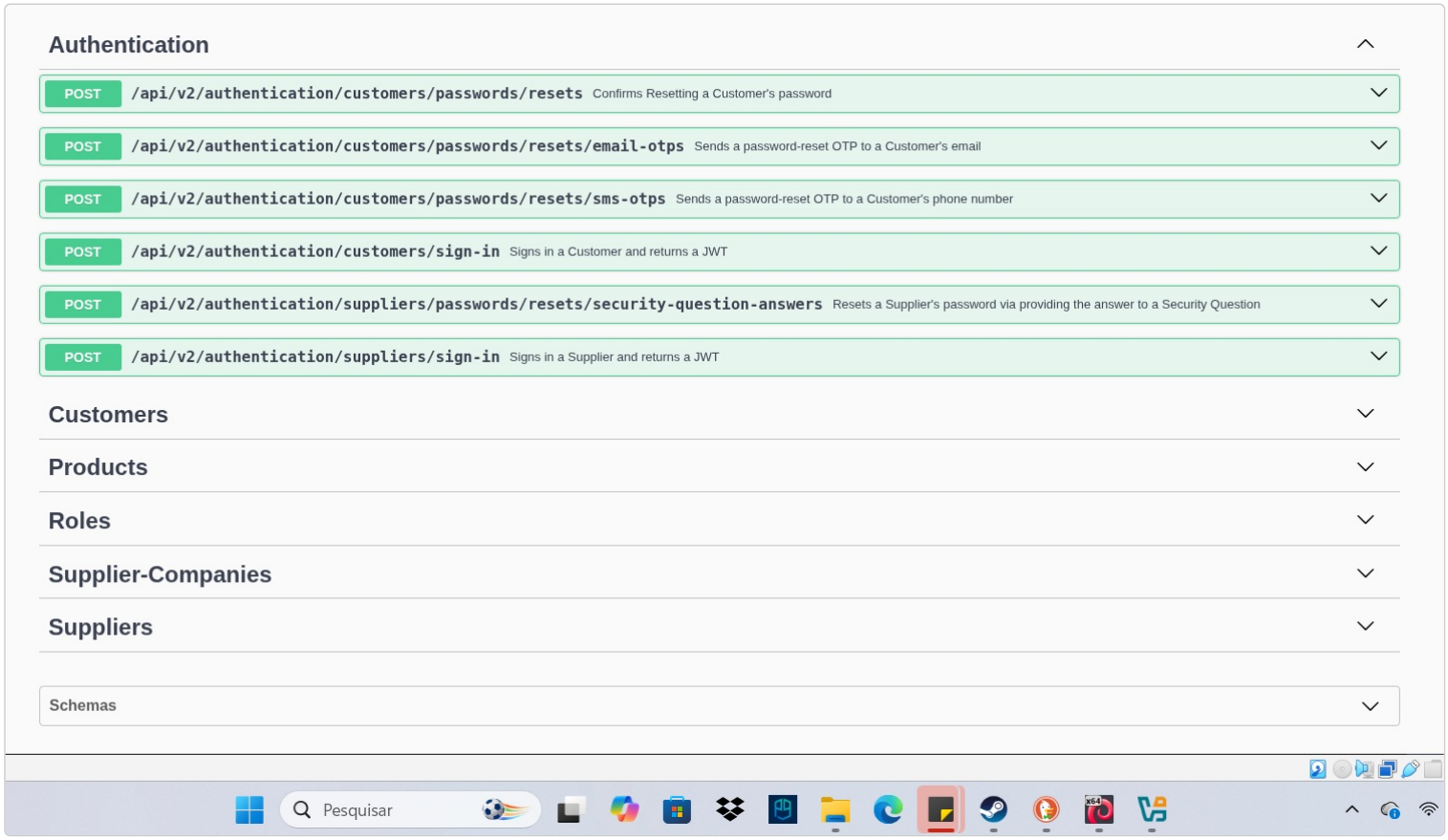


Figura 1 — Endpoints do grupo Authentication no Swagger, destacando o fluxo de reset de senha de fornecedores via resposta de pergunta de segurança (`/api/v2/authentication/suppliers/passwords/resets/security-question-answers`).

Endpoints relevantes identificados nesta fase:

- `POST /api/v2/authentication/customers/sign-in`
- `POST /api/v2/authentication/suppliers/sign-in`
- `POST /api/v2/authentication/suppliers/passwords/resets/security-question-answers`
- `GET /api/v2/roles/current-user`
- `GET /api/v2/suppliers`
- `PATCH /api/v2/suppliers/current-user`
- `GET /api/v2/suppliers/current-user/cv`

5. Fase 2 — Autenticação e Privilégios Excessivos ALTO

5.1 Autenticação com credenciais de teste

Utilizando as credenciais de teste fornecidas pelo laboratório (`htbpentester@hackthebox.com`), foi realizado um login através do endpoint de clientes, retornando um token JWT válido.

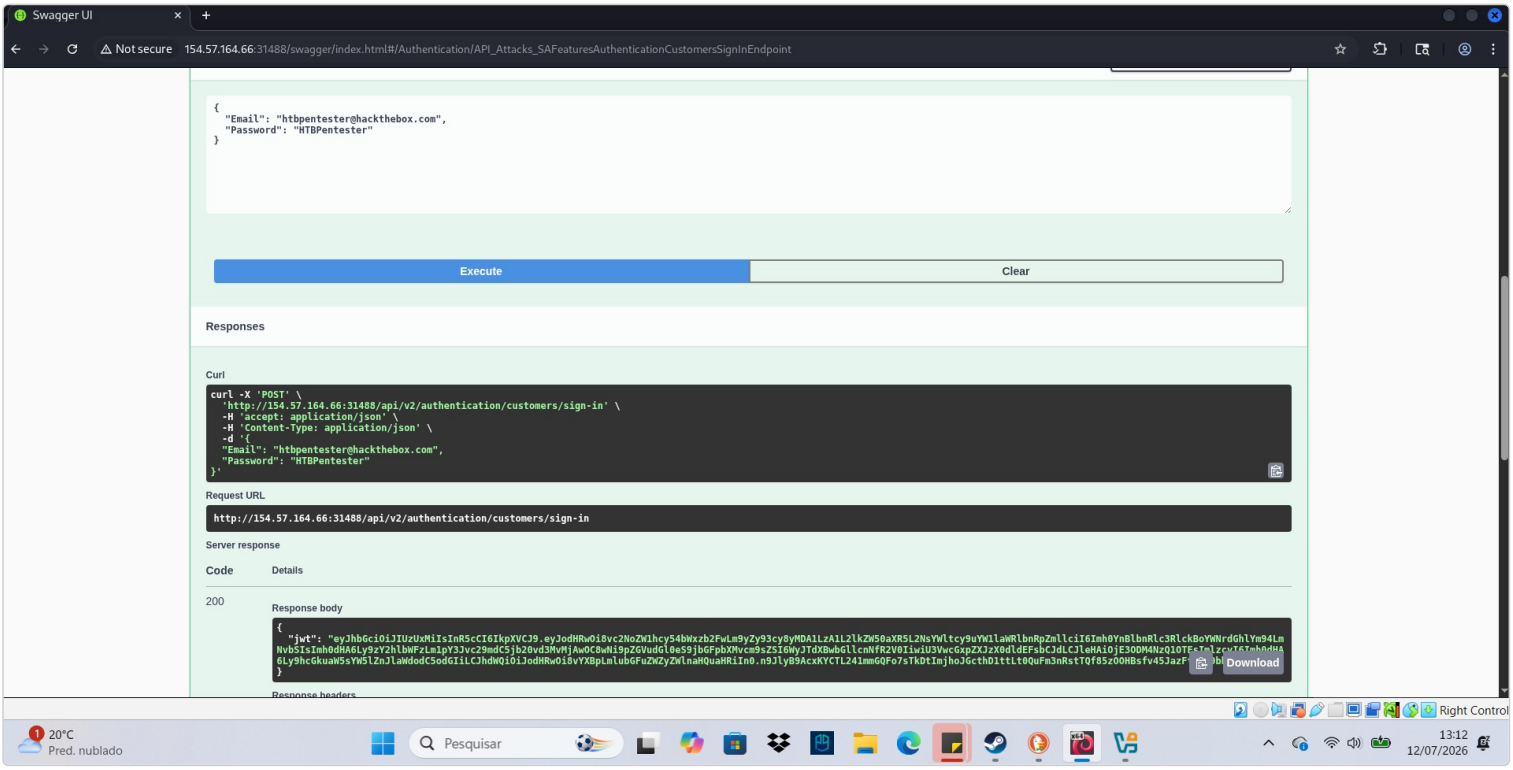


Figura 2 — Login bem-sucedido via `/authentication/customers/sign-in`, retornando um JWT no corpo da resposta (200 OK).

5.2 Autorização do token no Swagger

O JWT obtido foi inserido no diálogo de autorização do Swagger (esquema `JWTBearerAuth`), habilitando o envio automático do cabeçalho `Authorization: Bearer <token>` em todas as chamadas subsequentes.

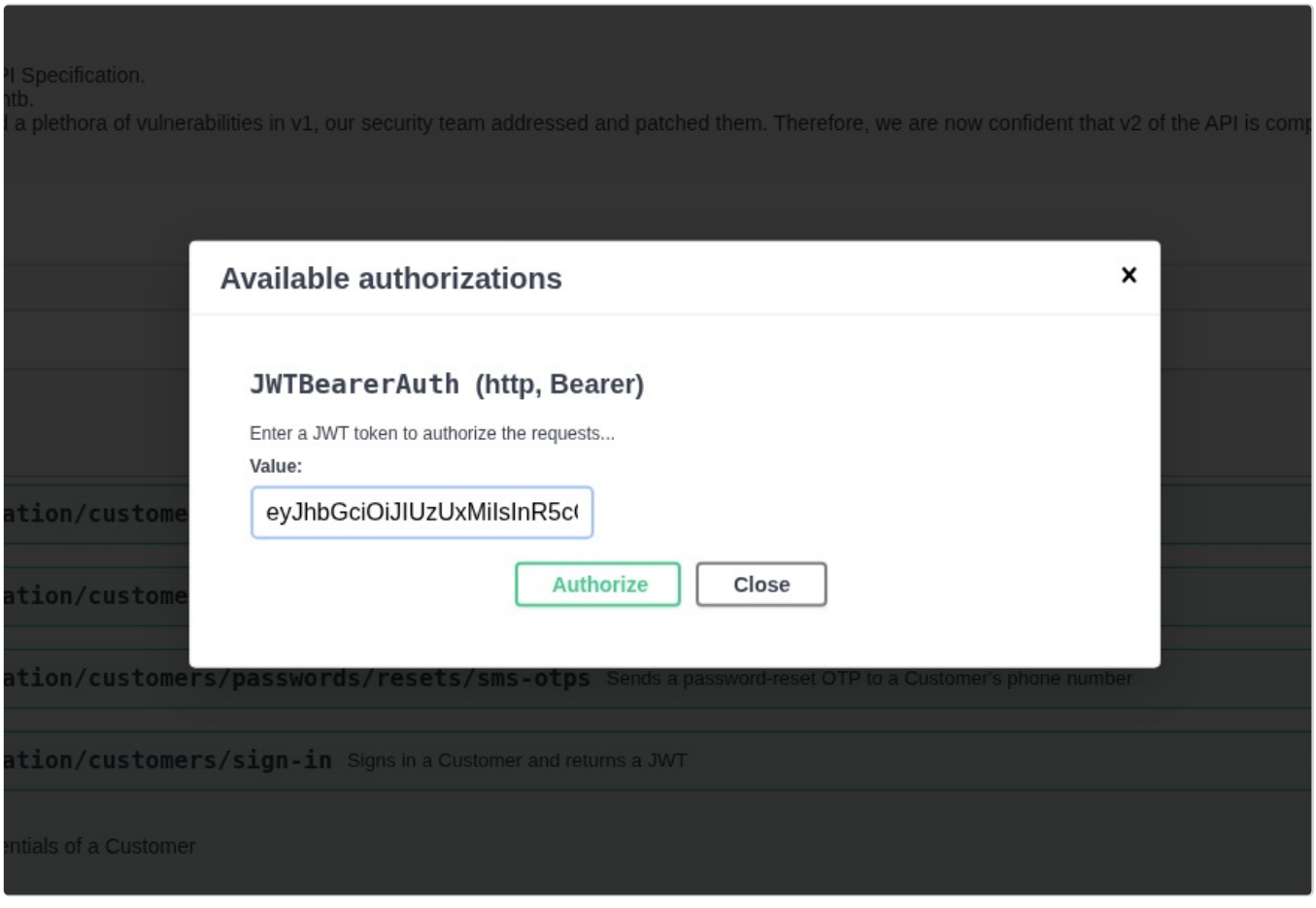


Figura 3 — Configuração do token JWT no Swagger UI para autorizar as próximas requisições.

5.3 Verificação de papéis (roles) do usuário

Consultando o endpoint `GET /api/v2/roles/current-user` com o token de cliente, observou-se que a conta possuía os papéis `Suppliers_Get` e `Suppliers_GetAll` — permissões que deveriam pertencer exclusivamente ao domínio de *Fornecedores*, e não a uma conta autenticada como *Cliente*. Trata-se de uma falha de **Broken Function Level Authorization (API5:2023)**: o controle de acesso não segrega corretamente as funções permitidas por tipo de conta.

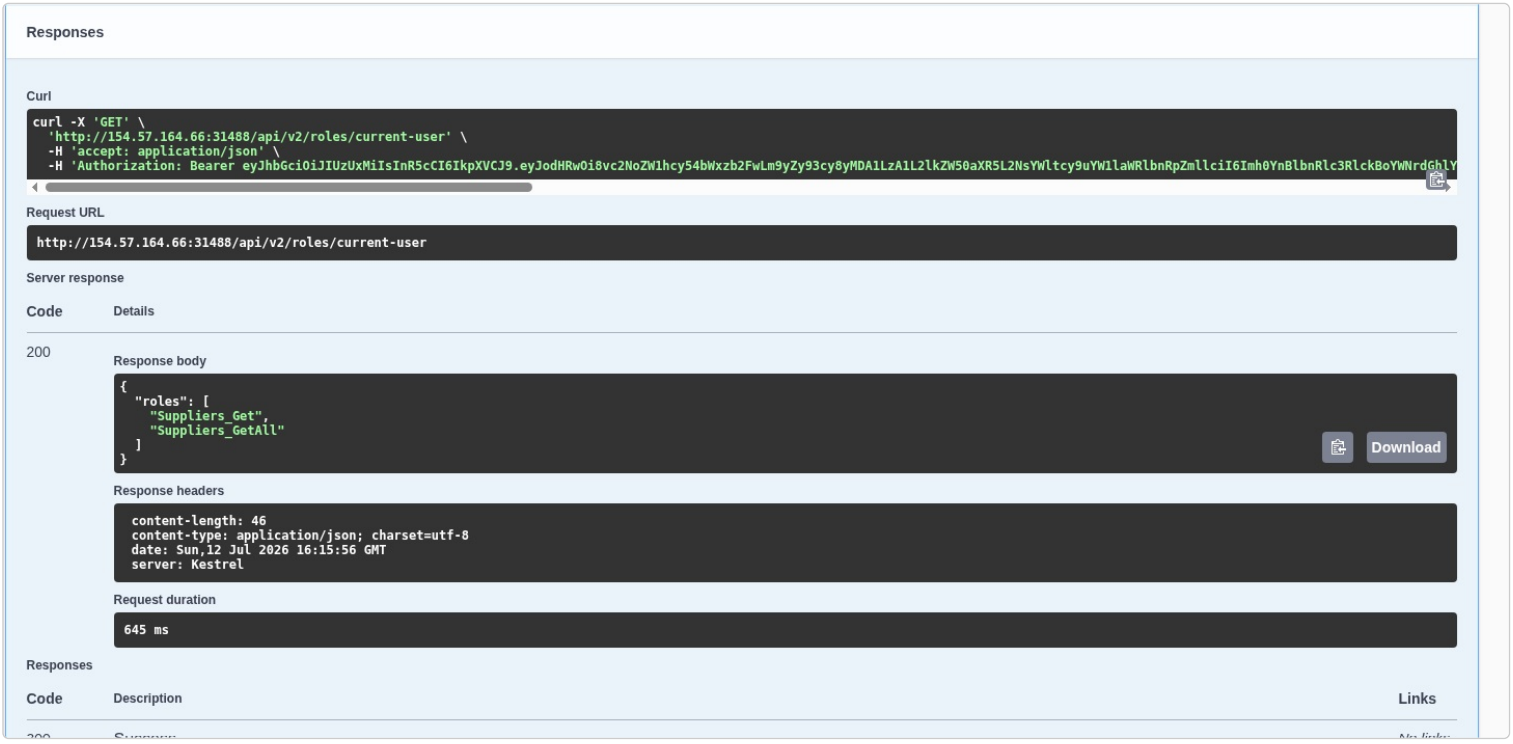


Figura 4 — Resposta do endpoint de papéis revelando permissões indevidas de fornecedor atribuídas à conta de cliente autenticada.

6. Fase 3 — Enumeração de Fornecedores (BOLA) ALTO

Com os papéis de fornecedor indevidamente concedidos, foi possível consultar `GET /api/v2/suppliers` e obter a listagem completa de todos os fornecedores cadastrados na plataforma — um claro caso de **Broken Object Level Authorization (API1:2023)**, já que nenhuma regra de propriedade (ownership) limitava o acesso aos próprios dados do usuário autenticado.

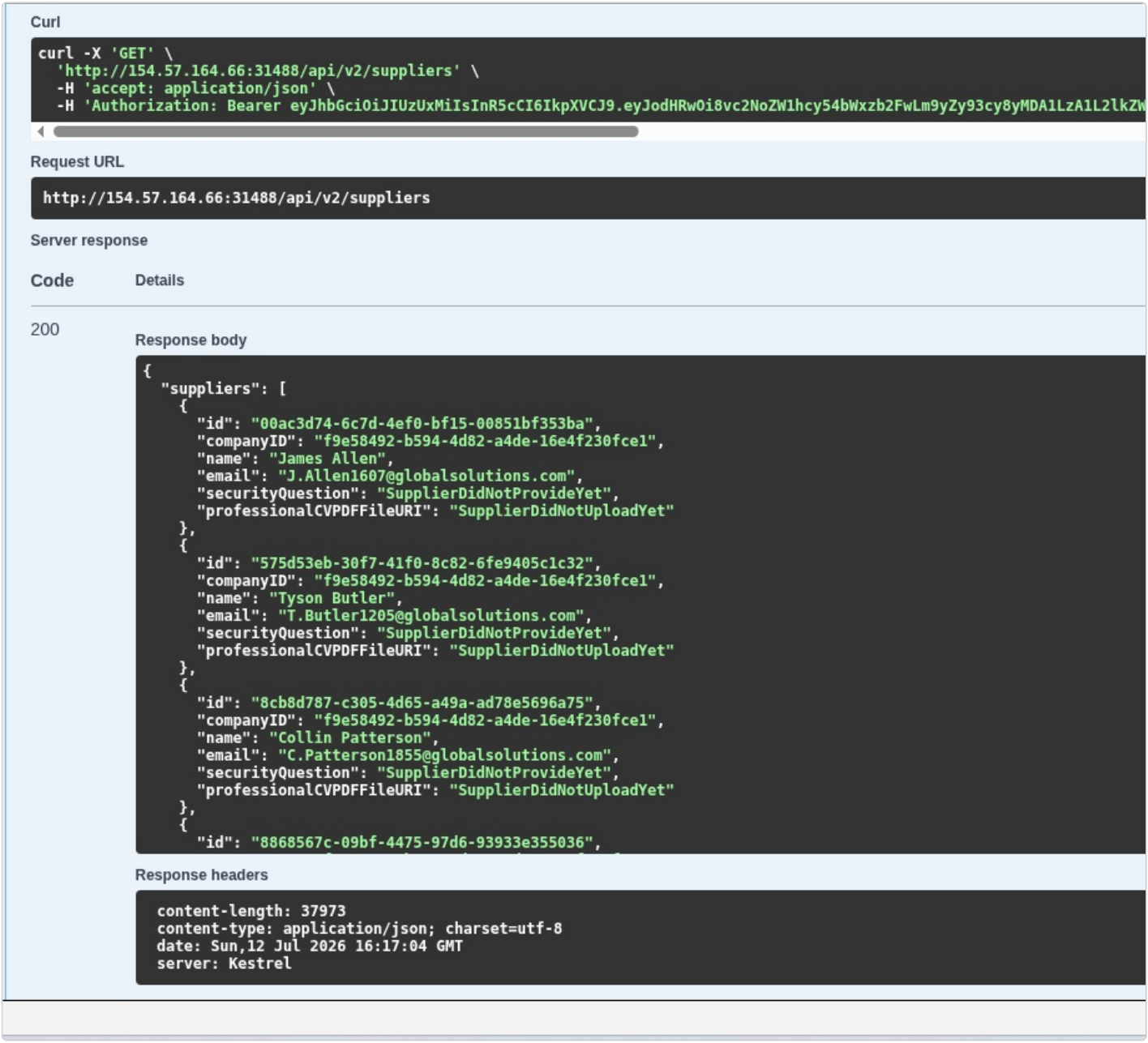


Figura 5 — Listagem completa de fornecedores, incluindo nome, e-mail, pergunta de segurança (`securityQuestion`) e URI do currículo.

> Muitos registros continham o valor `"SupplierDidNotProvideYet"` no campo `securityQuestion`, mas parte dos fornecedores já havia configurado uma pergunta real (ex.: "*What is your favorite color?*"). Esse subconjunto tornou-se o alvo do próximo passo.

Dado o volume elevado de registros retornados (dezenas de fornecedores em um único JSON), a filtragem dos registros que possuíam pergunta de segurança já configurada foi feita com o auxílio de um LLM (DeepSeek), agilizando a extração dos pares e-mail/pergunta relevantes para o ataque de força bruta.

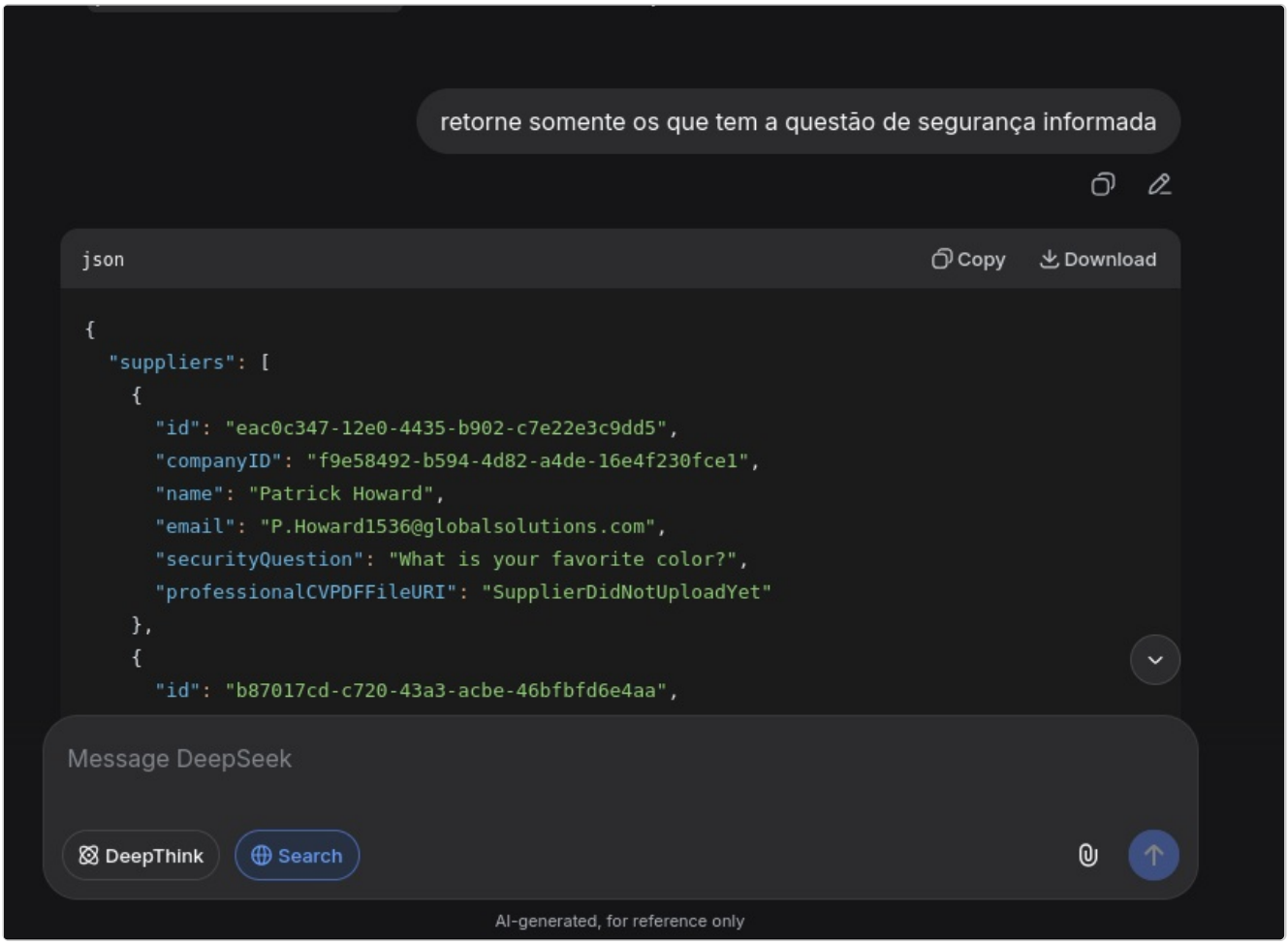


Figura 6 — Uso de um LLM para filtrar, dentro do JSON extenso, apenas os fornecedores que já haviam informado uma pergunta de segurança.

7. Fase 4 — Força Bruta na Resposta de Segurança CRÍTICO

Com uma lista de e-mails de fornecedores que possuíam a pergunta "*What is your favorite color?*" configurada, o passo seguinte foi validar manualmente o comportamento do endpoint de reset de senha, enviando uma resposta incorreta para confirmar o formato da requisição e da resposta de erro.

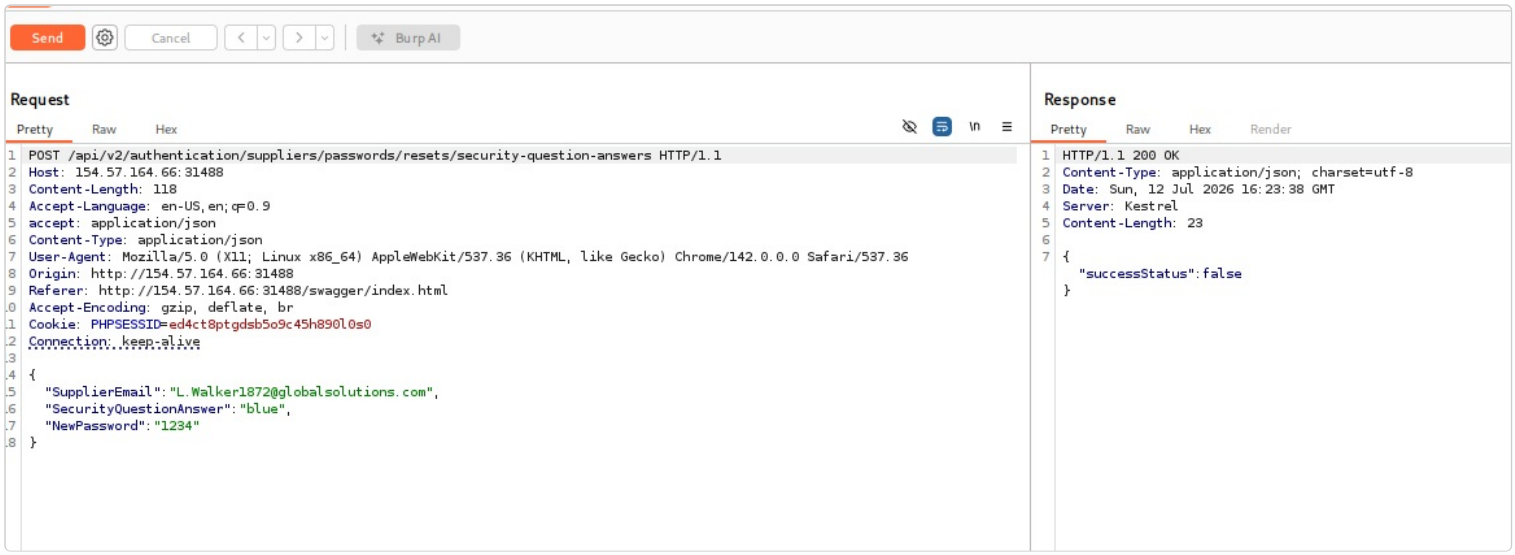


Figura 7 — Requisição manual via Burp Suite com resposta incorreta (`"blue"`), retornando `"successStatus": false`.

Como o endpoint não implementava nenhum controle de taxa (rate limiting) ou bloqueio de conta após tentativas malsucedidas, foi montado um ataque de força bruta com **ffuf** em modo `clusterbomb`, combinando uma wordlist de e-mails de fornecedores com uma wordlist de nomes de cores comuns em inglês, usando o filtro de resposta pelo código HTTP para distinguir sucesso (200 com `successStatus: true`) de falha.

```
ffuf -u http://154.57.164.66:31488/api/v2/authentication/suppliers/passwords/resets/security-question-answers \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"SupplierEmail":"USER", "SecurityQuestionAnswer":"COLOR", "NewPassword":"123456"}' \
  -w users.txt:USER \
  -w colors.txt:COLOR \
  -mode clusterbomb \
  -fr "false" -c
```

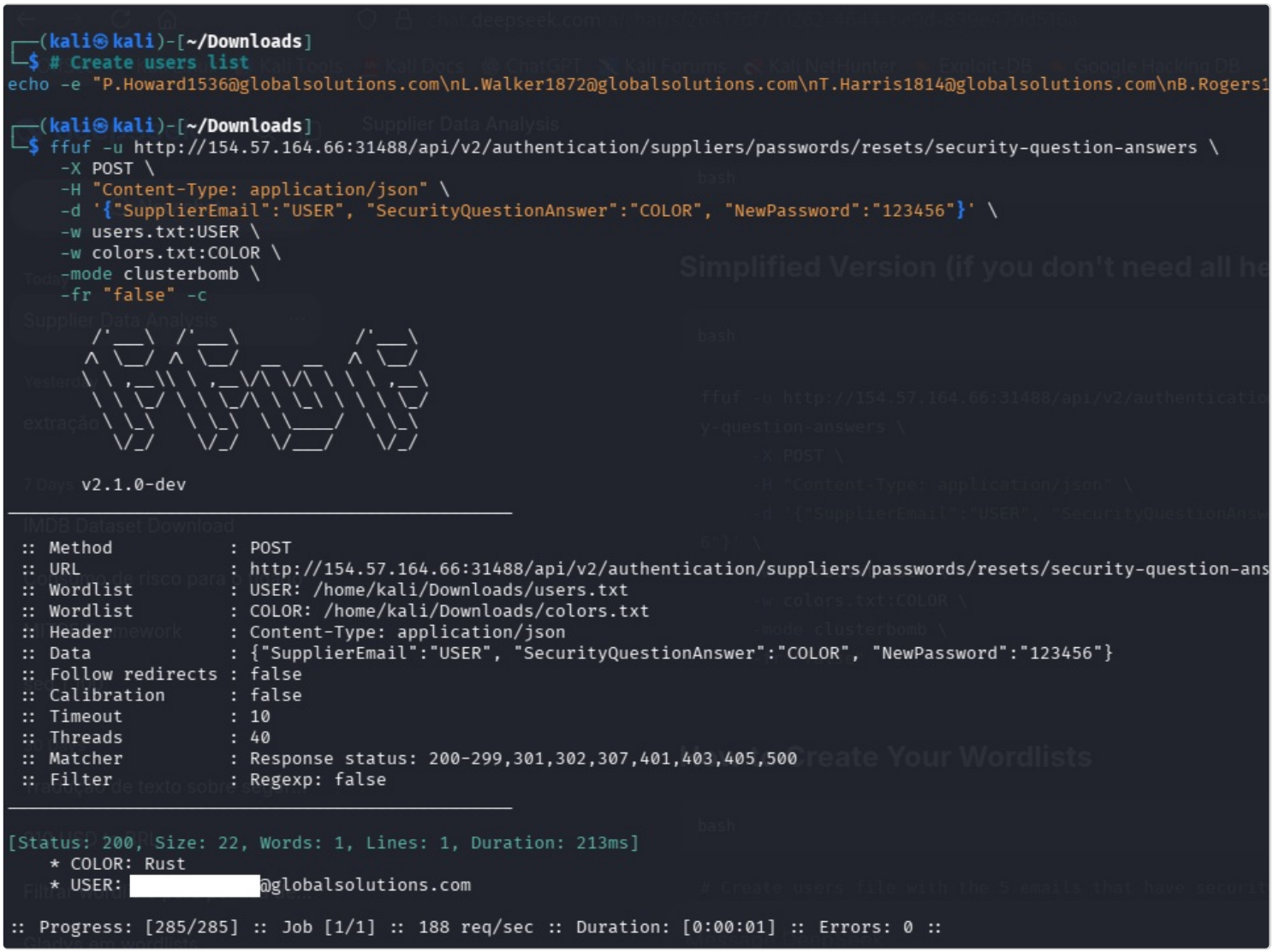


Figura 8 — Execução do ffuf combinando e-mails e cores candidatas. O resultado `[Status: 200]` identifica a combinação correta de e-mail e resposta de segurança (cor: Rust).

Impacto: a ausência de limitação de tentativas em um endpoint de recuperação de senha baseado em pergunta de segurança — um mecanismo com espaço de respostas naturalmente pequeno (nomes de cores) — tornou o ataque de força bruta trivial e rápido (285 combinações testadas em cerca de 1 segundo), resultando em comprometimento total de conta.

8. Fase 5 — Account Takeover via Reset de Senha CRÍTICO

Com a resposta correta identificada (`"Rust"`), a requisição de reset foi reproduzida manualmente no Burp Suite, definindo uma nova senha de escolha do atacante para a conta do fornecedor. A resposta confirmou o sucesso da operação.

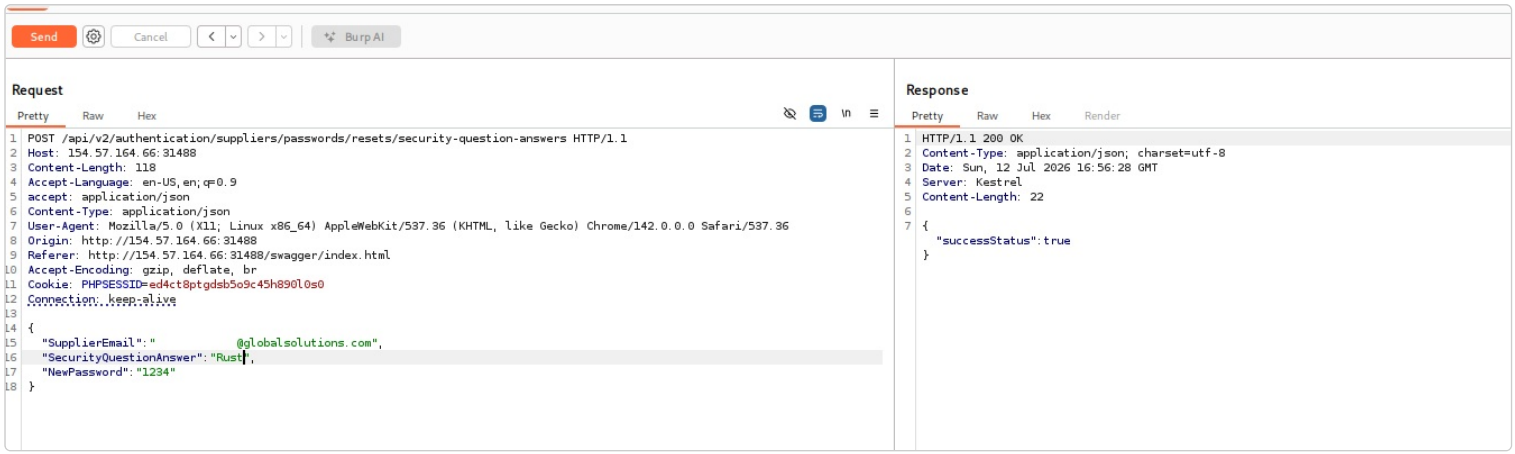


Figura 9 — Reset de senha bem-sucedido (`"successStatus": true`) utilizando a resposta de segurança correta descoberta via força bruta.

Com a nova senha definida, foi possível autenticar-se na conta do fornecedor comprometido através de `POST /api/v2/authentication/suppliers/sign-in` , obtendo um novo token JWT com o contexto de fornecedor. Curiosamente, ao consultar os papéis desta conta específica, o sistema retornou que o usuário não possuía nenhum papel atribuído — evidenciando uma inconsistência adicional no modelo de autorização da aplicação, onde o acesso às funcionalidades de fornecedor independe da verificação formal de papéis.

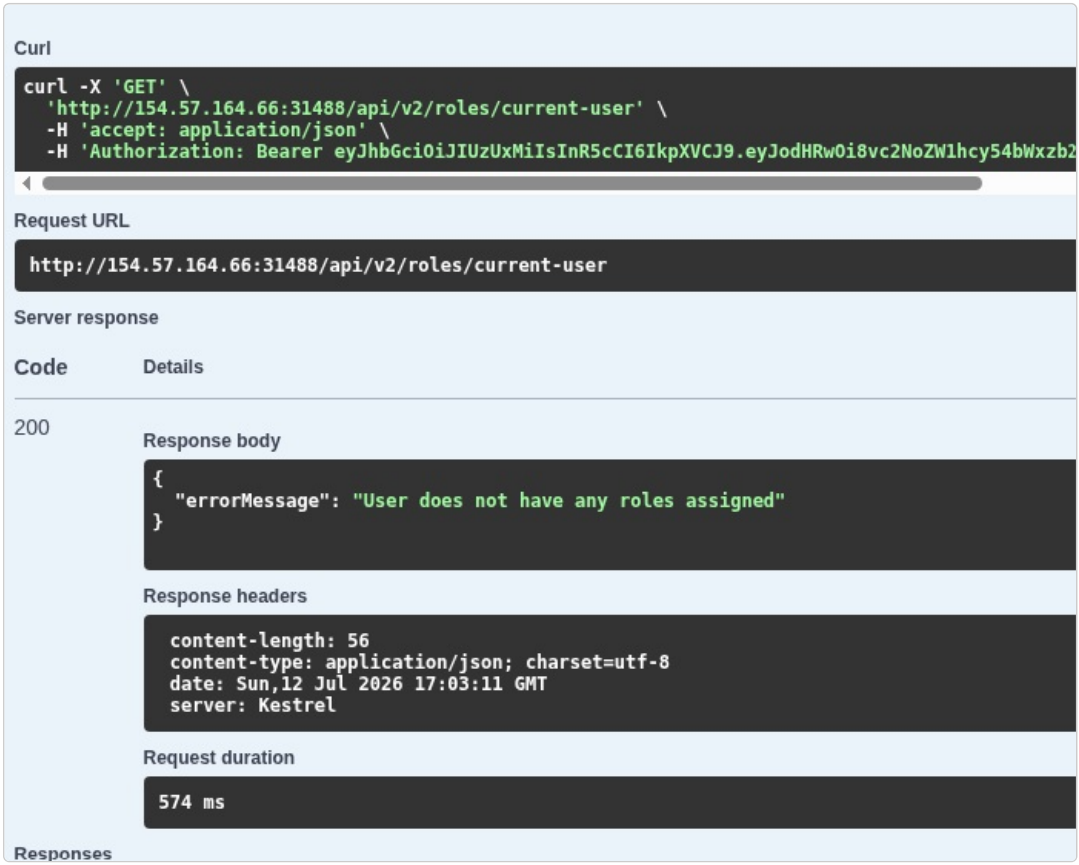


Figura 10 — Consulta de papéis da conta de fornecedor comprometida, retornando ausência de papéis atribuídos — a autorização de funcionalidades não depende deste controle.

9. Fase 6 — Local File Inclusion via Upload de CV CRÍTICO

Já autenticado como fornecedor, o endpoint `PATCH /api/v2/suppliers/current-user` permite atualizar diversos campos do perfil, entre eles `ProfessionalCVPDFFileURI` — o próprio exemplo da documentação Swagger sugeria o valor `"file:///etc/passwd"`, um forte indício de que o campo não realiza validação de esquema/protocolo da URI fornecida.

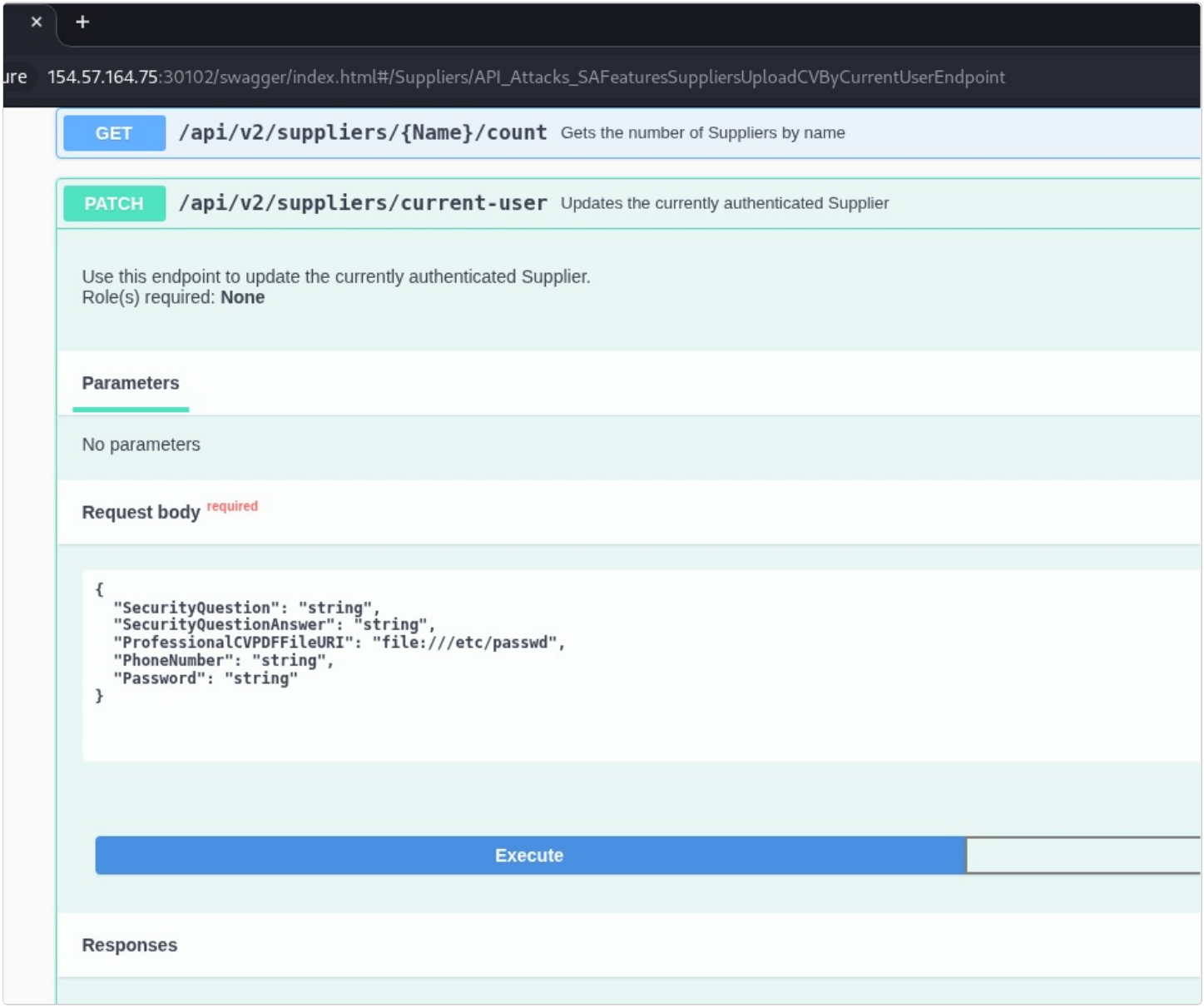


Figura 11 — Corpo da requisição PATCH no Swagger, com o campo `ProfessionalCVPDFFileURI` aceitando o esquema `file://`.

Antes deste teste, o perfil do fornecedor possuía um CV padrão apontando para um arquivo interno da aplicação (`file:///app/wwwroot/SupplierCVs/empty.pdf`), confirmando que o backend de fato resolve e lê o conteúdo do arquivo referenciado pela URI armazenada, em vez de apenas armazenar a string.

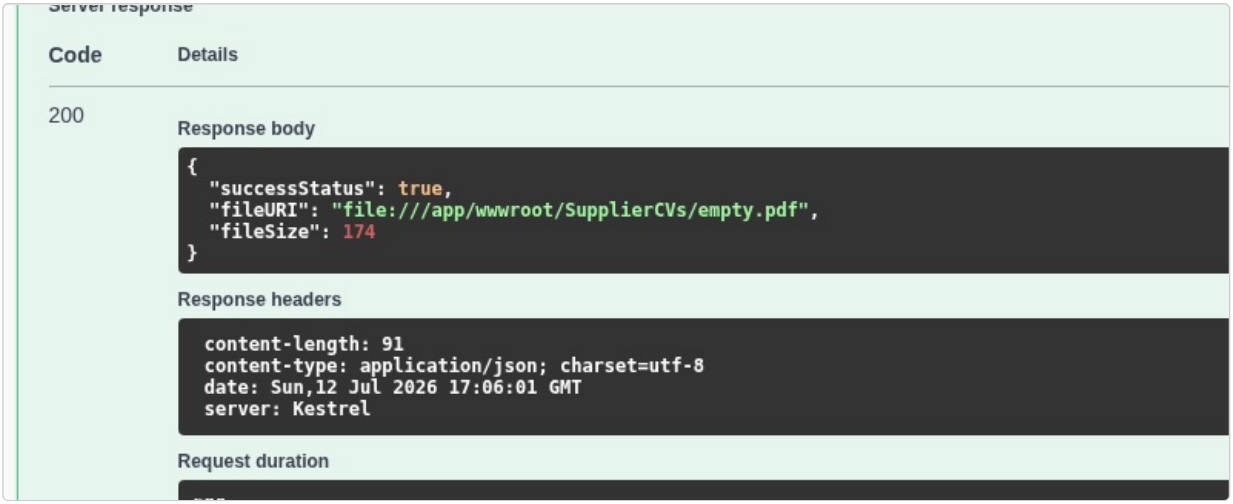


Figura 12 — Estado inicial do campo de CV, apontando para um arquivo interno do servidor de aplicação, confirmando que a URI é resolvida no sistema de arquivos local.

Explorando esse comportamento, o campo foi sobrescrito com um caminho arbitrário no sistema de arquivos do servidor via o esquema `file://`, e a requisição foi enviada com sucesso (200 OK).

```
curl -X 'PATCH' \
  'http://154.57.164.75:30102/api/v2/suppliers/current-user' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer <JWT_DO_FORNECEDOR>' \
  -H 'Content-Type: application/json' \
  -d '{
  "SecurityQuestion": "string",
  "SecurityQuestionAnswer": "string",
  "ProfessionalCVPDFFileURI": "file:///caminho/para/o/arquivo/da/flag",
  "PhoneNumber": "string",
  "Password": "string"
}'
```

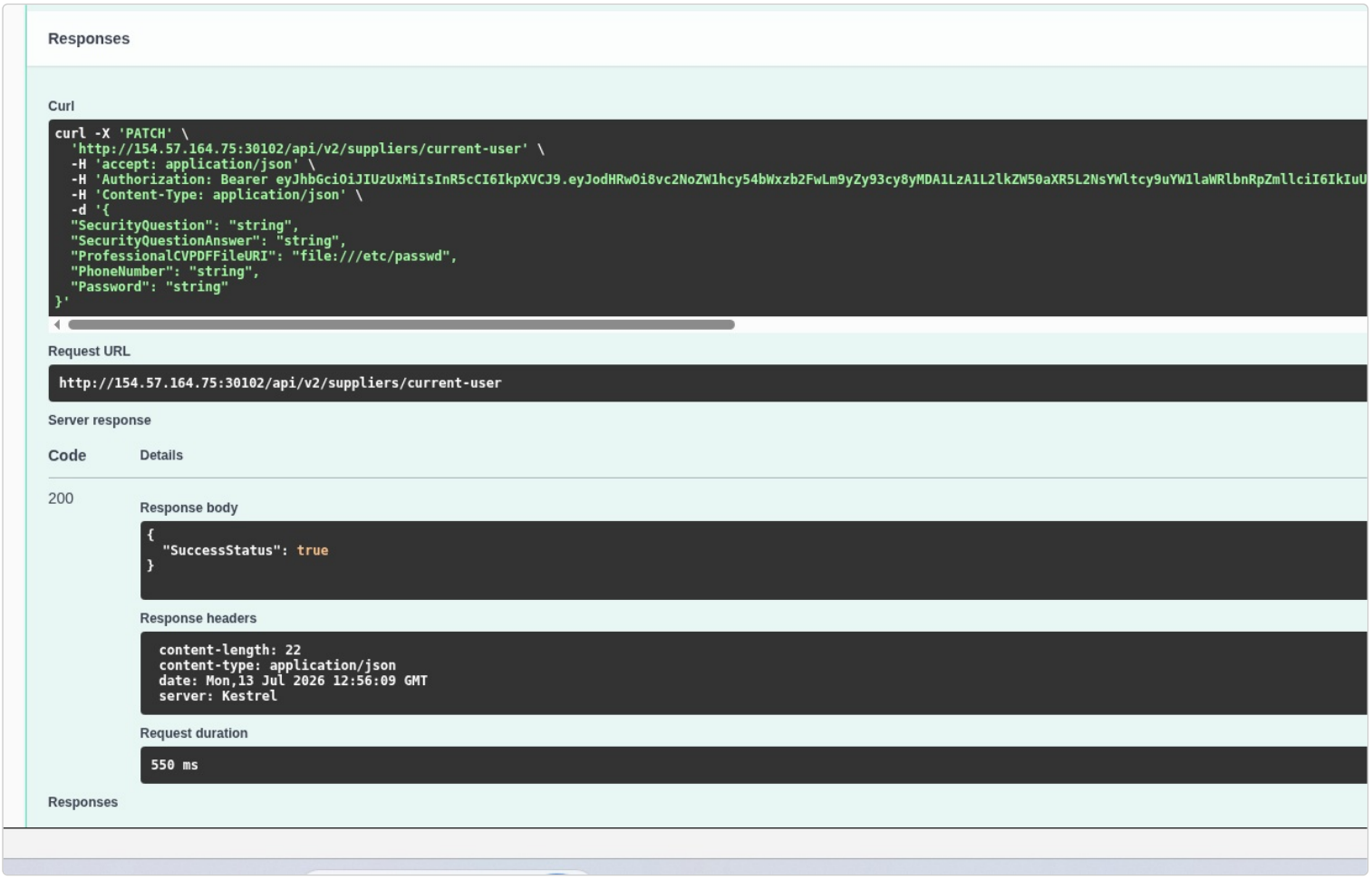


Figura 13 — Requisição PATCH aplicada com sucesso, sobrescrevendo a URI do CV para apontar a um arquivo arbitrário do servidor.

Em seguida, o endpoint `GET /api/v2/suppliers/current-user/cv` foi consultado para recuperar o conteúdo do "currículo" — que, na realidade, retornou o conteúdo do arquivo arbitrário lido do disco do servidor, codificado em Base64.

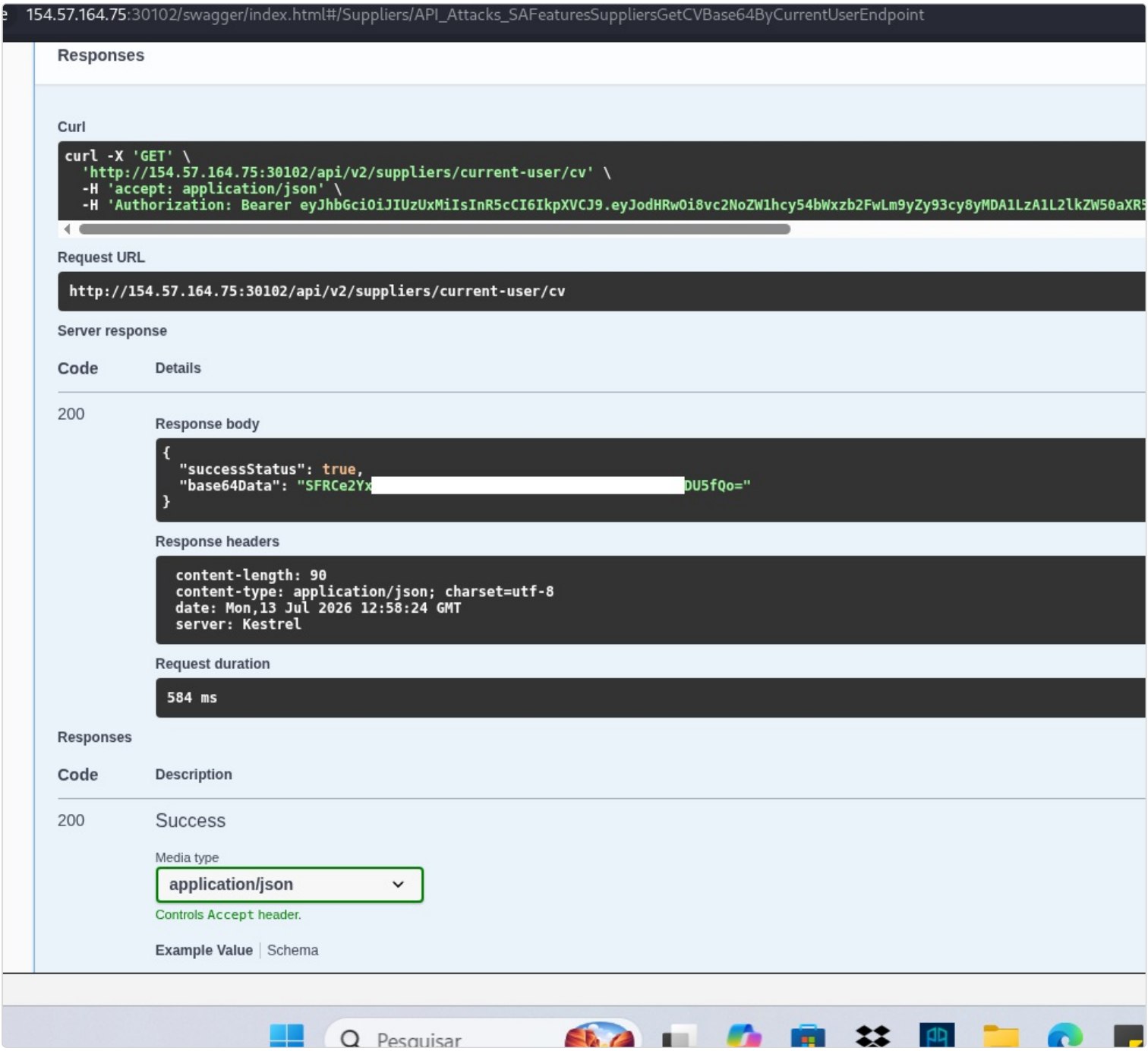


Figura 14 — Resposta contendo o campo `base64Data` com o conteúdo do arquivo lido do servidor via LFI.

Decodificando o valor Base64 (via CyberChef), o conteúdo revelou a flag do desafio, confirmando a exploração completa da cadeia de vulnerabilidades.

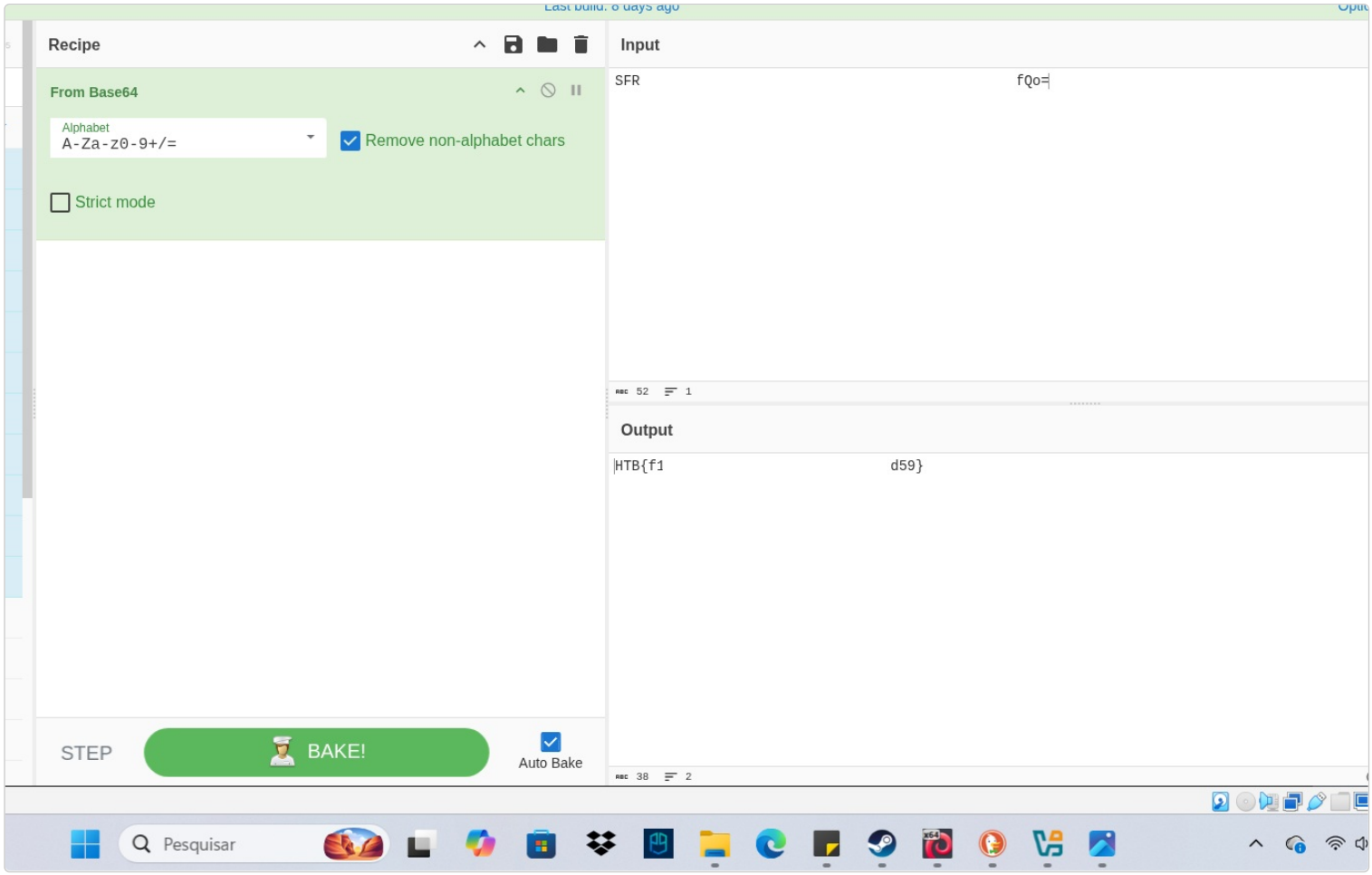


Figura 15 — Decodificação Base64 no CyberChef, revelando a flag no formato `HTB{...}`.

10. Resumo das Vulnerabilidades e Riscos

#	Vulnerabilidade	Classificação OWASP API Top 10	Severidade	Descrição resumida
1	Privilégios excessivos concedidos a conta de cliente	API5:2023 — Broken Function Level Authorization	Alta	Conta autenticada como cliente recebeu papéis de fornecedor, habilitando acesso a funcionalidades de outro domínio de negócio.
2	Exposição total da listagem de fornecedores	API1:2023 — Broken Object Level Authorization	Alta	Endpoint `/api/v2/suppliers` retornou dados de todos os fornecedores, incluindo perguntas de segurança configuradas.
3	Ausência de rate limiting no reset de senha	API2:2023 — Broken Authentication	Crítica	Permitiu força bruta bem-sucedida da resposta de segurança e reset de senha sem qualquer bloqueio ou atraso progressivo.
4	Account Takeover de fornecedor	API2:2023 — Broken Authentication	Crítica	Consequência direta da falha #3: tomada de controle total de uma conta de fornecedor legítima.
5	Local File Inclusion via `ProfessionalCVPDFFileURI`	API8:2023 — Security Misconfiguration	Crítica	Campo de URI de upload de CV não valida o esquema/protocolo, permitindo leitura de arquivos arbitrários do servidor via `file://` .

11. Recomendações de Correção

1. **Autorização por função (RBAC) consistente:** revisar a atribuição de papéis no momento do cadastro/login, garantindo que contas de cliente jamais recebam papéis de fornecedor, e reforçar a verificação de papéis em toda rota sensível do lado do servidor (nunca confiar apenas no token).
2. **Controle de acesso a nível de objeto (BOLA):** a listagem de fornecedores deve ser restrita a administradores ou filtrada por contexto de propriedade, nunca expondo campos sensíveis como perguntas de segurança em respostas de listagem.
3. **Rate limiting e bloqueio de conta:** implementar limitação de tentativas (ex.: 5 tentativas por hora por conta/IP), CAPTCHA e bloqueio temporário no endpoint de reset de senha por pergunta de segurança.
4. **Eliminar perguntas de segurança como mecanismo único de recuperação:** substituir por OTP via e-mail/SMS com expiração curta e múltiplos fatores, já que perguntas de segurança possuem espaço de respostas previsível e são um mecanismo legado inseguro.
5. **Validação estrita de esquema de URI:** o campo de upload de CV deve aceitar apenas referências internas geradas pelo próprio backend (ex.: identificadores opacos), nunca uma URI arbitrária fornecida pelo cliente. Caso uma URI externa seja necessária, restringir a lista de esquemas permitidos (allowlist) a `https://` e validar o host de destino, bloqueando `file://`, `ftp://` e endereços internos (mitigação de SSRF/LFI).
6. **Princípio do menor privilégio para o processo da aplicação:** mesmo com validação de entrada, o processo do servidor não deveria ter permissão de leitura sobre arquivos sensíveis do sistema operacional.

12. Conclusão

O Skills Assessment do módulo **API Attacks** demonstrou, na prática, como falhas aparentemente isoladas de autorização e validação de entrada podem ser encadeadas para resultar em comprometimento total de dados sensíveis. Partindo de um simples excesso de privilégios concedido a uma conta comum, foi possível enumerar toda a base de fornecedores, quebrar por força bruta um mecanismo de recuperação de senha mal projetado, assumir o controle de uma conta legítima e, por fim, explorar uma falha de Local File Inclusion para ler arquivos arbitrários do servidor e capturar a flag do desafio.

Este exercício reforça a importância de aplicar os princípios do **OWASP API Security Top 10** em todas as camadas de uma API — autenticação, autorização a nível de função e de objeto, e validação rigorosa de qualquer entrada controlada pelo cliente — especialmente quando essa entrada é usada para operações de leitura/escrita em sistemas de arquivos.